SmartHR

AI・可観測性・蛇口 



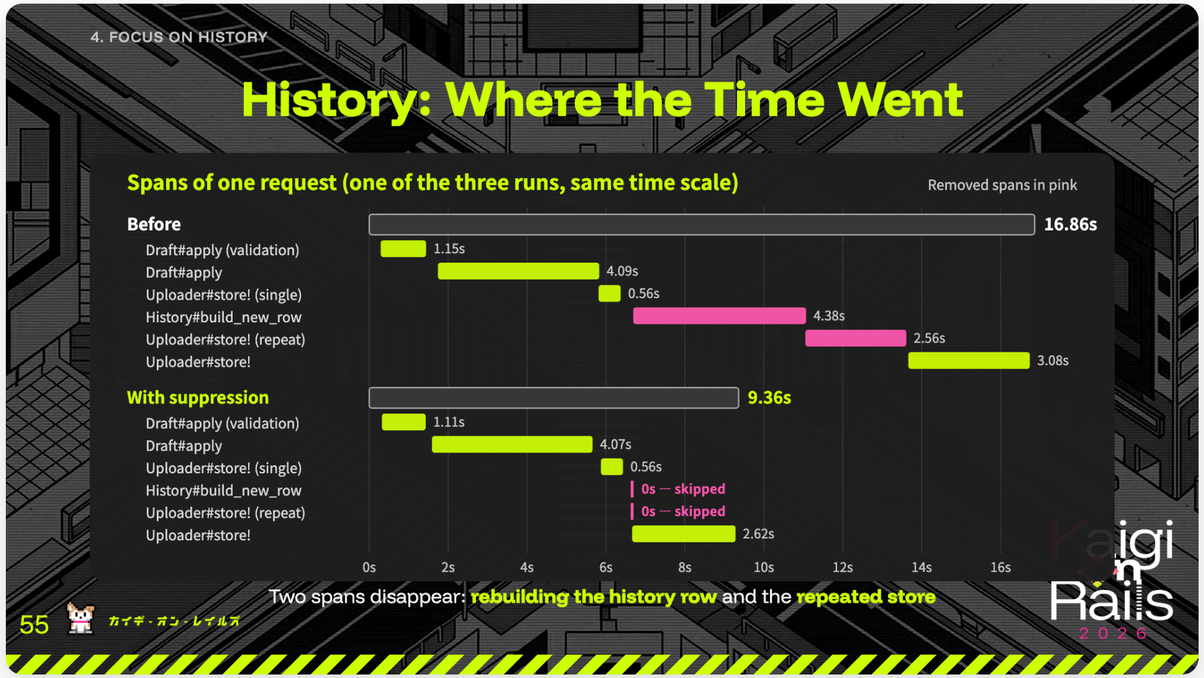
SmartHR 技術基盤部 / Fukuoka.rb
近藤 宇智朗 (@udzura)

© SmartHR, Inc.

Kaigi on Rails 2026 Recap

- **Zen and the Art of File Upload Maintenance**
- CarrierWave との戦いの記録...
- 信頼性、アップグレード、そしてパフォーマンス

こんな計測結果、ありましたよね



横棒の一本一本が span

出典: Zen and the Art of File Upload Maintenance (Kaigi on Rails 2026) p.55 — https://udzura.jp/slides/2026/kaigionrails/

どうやって測ったの？

orange_tap

- メソッドの処理時間・呼び出しツリーの可視化ツール
- 利用技術: `TracePoint`(Ruby), OpenTelemetry※, OTLP/JSON
- Pure Ruby 製。RubyGems.org で公開中

※ OpenTelemetry: トレース・メトリクス・ログを扱う、ベンダー中立な可観測性の標準（CNCF）。
OTLP はそのデータ形式・転送プロトコル

名前の由来

- SystemTap っぽい名前にしたかった
  - Linux の定番トレーサ
- じゃあ「〇〇tap」... tap...

RubyKaigi 2025 Matsuyama

Orange tap...

🍊🚰

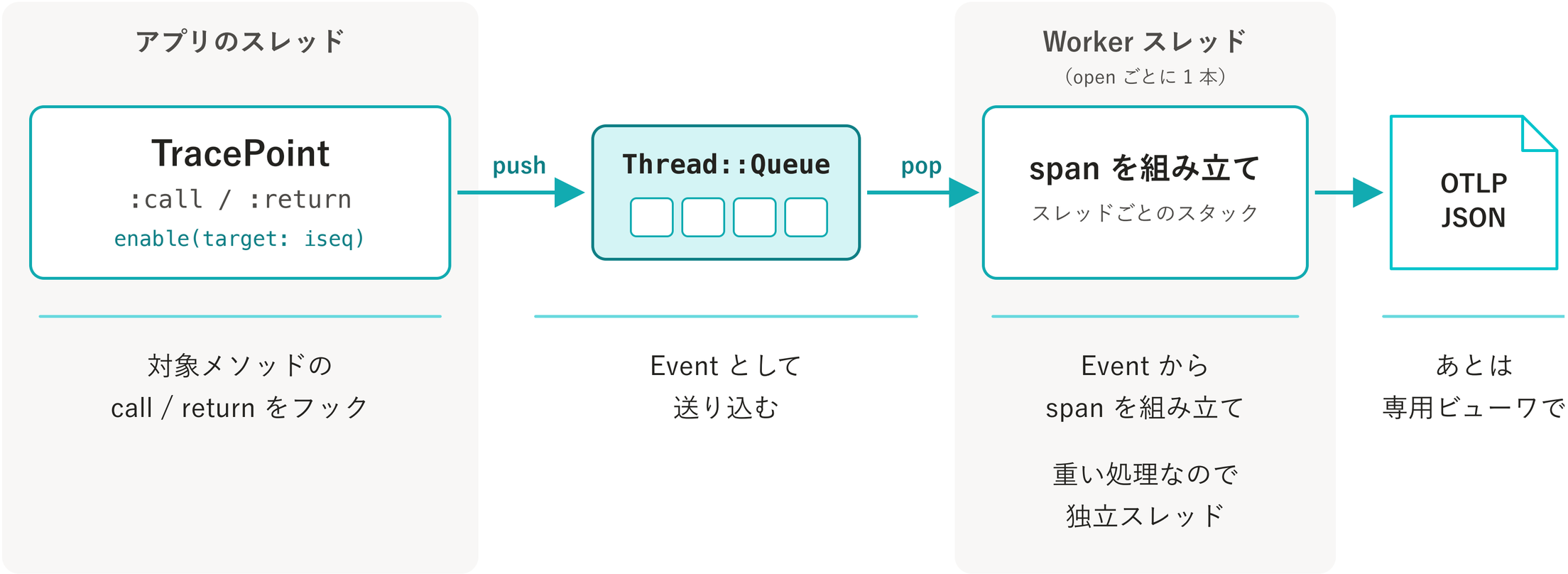


使い方

```ruby
# Uploader / Storage はデモ用の小さなクラス
OrangeTap.trace_all_instance_methods(Uploader)
OrangeTap.trace_method("Storage#head", "Storage#put")

OrangeTap.open("POST /avatar") do
  Uploader.new.store!("me.png")
end
# => "out/orange_tap-1f0016....json"
```

内部実装: 全体像


アプリのスレッド
TracePoint
:call / :return
enable(target: iseq)
push
Thread::Queue
pop
Worker スレッド
（open ごとに 1 本）
span を組み立て
スレッドごとのスタック
OTLP
JSON
対象メソッドの
call / return をフック
Event として
送り込む
Event から
span を組み立て
重い処理なので
独立スレッド
あとは
専用ビューワで

どう可視化するの？

otel-oneshot

- OTLP/JSON を **ASCII のタイムライン** にする CLI（Go 製）
- 対話 UI なし、1 回描いて終わり
  - パイプ・CI・AI エージェントでそのまま使える
- `--only` / `--hide` / `--sort duration` で絞り込み
- `brew install --cask udzura/tap/otel-oneshot`

otel-oneshot で見る

```
$ otel-oneshot out/orange_tap-1f0016....json --hide '^tid='
span                                0ms                  200.65ms                401.31ms
└─ POST /avatar                     ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓ 401.31ms
   └─ Uploader#store!               ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓ 401.14ms
      ├─ Uploader#resize_all        ▓▓▓▓▓▓▓▓▓▓▓▓▓                                             82.46ms
      ├─ Uploader#store_version!            ▓▓▓▓▓▓▓▓▓▓▓▓▓                                     80.11ms
      │  ├─ Storage#head                    ▓▓▓▓                                              25.02ms
      │  └─ Storage#put                        ▓▓▓▓▓▓▓▓▓                                     55.04ms
      ├─ Uploader#store_version!                     ▓▓▓▓▓▓▓▓▓▓▓▓▓                            80.09ms
      │  ├─ Storage#head                             ▓▓▓▓▓                                    25.03ms
      │  └─ Storage#put                                  ▓▓▓▓▓▓▓▓▓                           55.03ms
      ├─ Uploader#store_version!                              ▓▓▓▓▓▓▓▓▓▓▓▓▓                   79.8ms
      │  ├─ Storage#head                                      ▓▓▓▓▓                            25.06ms
      │  └─ Storage#put                                          ▓▓▓▓▓▓▓▓▓▓                   54.68ms
      └─ Uploader#store_version!                                        ▓▓▓▓▓▓▓▓▓▓▓▓▓          78.63ms
         ├─ Storage#head                                                ▓▓▓▓                   23.51ms
         └─ Storage#put                                                    ▓▓▓▓▓▓▓▓▓           55.05ms
```

同じ形が並んだら N+1

```
$ otel-oneshot trace.json --only 'Storage#put'
├─ Storage#put  ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒                                          55.04ms
├─ Storage#put                 ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒                          55.03ms
├─ Storage#put                                ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒          54.68ms
└─ Storage#put                                              ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒ 55.05ms
```

- `--only` で見たい span だけ残す
- `--sort duration --top-n` で遅い順に絞る

人間が使うだけじゃない

AI エージェントに使わせる

- 計測 → 描画 → 読解 の手順を **Skill**※ にまとめた
- Rails では `around_action` + `OrangeTap.record` で仕込む
  - spec の実行中に仕込む手もある

※ Skill: AI エージェント（Claude Code など）に読ませる手順書。
「こういう作業はこの手順で」をリポジトリに置いておく

AI に使わせた例

Claude Code とのセッションより

ファイルアップロード時の前処理のあるなしによるパフォーマンスの違いを計測したいです。orange-tap の使い方を覚えてる？

Skills を実行しました /orange-tap-trace ›

はい、覚えています（プロジェクトの `orange-tap-trace` スキルに手順があります）。

手順の要点

1. 対象のコントローラーに `around_action` を一時的に追加する。`Rails.env.development?` `ENV["ORANGE_TAP"] == "true"` のガードを付け、中で `OrangeTap.record(...) { yield`
2. 計測するメソッドは `trace_method` か `trace_all_instance_methods(Klass)` で指定する。`trace_all_app_methods: true` を使うと gem 内部まで全部拾えますが、fog や Net::H しまで記録されるので、トレースが GB 単位になることがあります。

Measurement is still running (5 conditions × 3 runs, roughly 6–8 minutes). I'll put the results together once the completion notification arrives.

実行済み 1 件のコマンド（1件失敗）, 完了しました 1件のバックグラウンドコマンド ›

16 回すべて成功し、後片付け（Flipper の復元と切り替え用ファイルの削除）も済みました。versions=off でも残る 7 秒の内訳を確認します。

Rendered E and A create traces ›

計測が終わりました。今回の申請では、**かかっている時間のほとんどは version 画像の生成で、EXIF 処理はほぼ効いていません。**

結果（各条件 3 回の中央値、ローカル環境）

条件	upload_file	create_by_applicant	全部ありとの差（create）
A. 全部あり（[redacted]、現状の設定）	1.81s	**17.69s**	—
B. 全部あり（[redacted] を ON）	1.78s	16.80s	−0.9s

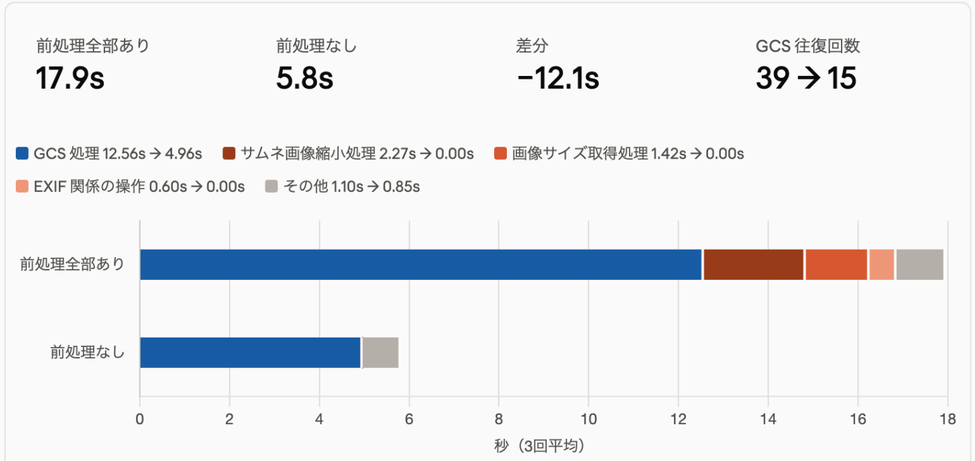
AI に使わせた例（続き）

Claude Code とのセッションより

create_by_applicant の GCS 通信

	全部あり（J）	version 生成なし（L、3 回の範囲）	差
通信の合計	**39 回 / 12.2s**	**15 回 / 5.08〜5.22s**	**−24 回 /**
put_object	11 回 / 4.63s	3 回 / 1.60〜1.65s	−8 回
copy_object	11 回 / 2.62s	3 回 / 0.71〜0.78s	−8 回
delete_object	11 回 / 3.16s	3 回 / 0.87〜0.88s	−8 回
get_object	3 回 / 1.39s	3 回 / 1.47〜1.62s	±0
get_object_metadata	3 回 / 0.40s	3 回 / 0.34〜0.38s	±0
リクエスト全体（実測の中央値）	17.93s	7.33s	−10.6s

図を作成しました



I regrouped the categories into the five you listed and redrew the chart above. Here's the same breakdown as a table:

Category	Full preprocessing	No preprocessing	Difference	Share (full preprocessing)
GCS operations	12.56s	4.96s	−7.60s	70%

学び: AI 時代こそ可観測性

- Before AI: アプリに可観測性を入れても、
  人間が使いこなしきれなかった
- AI は可観測性が大好き、というか **前提**
- debug-mcp※ も似た発想でしたね

※ Kaigi on Rails 2026「AIエージェントのデバッグ能力を向上させよう ― AI時代にRailsが輝く理由」（rira100000000 さん）。
debug gem を MCP で AI エージェントにつなぎ、ブレークポイントで止めて実行中の状態を見せる

まとめ

- **orange_tap**: メソッドの呼び出しをコールツリーとして記録
- **otel-oneshot**: タイムラインをターミナルで可視化
- 人間だけでなくAIも使える可観測性ツールを作ろう！

github.com/udzura/orange_tap

github.com/udzura/otel-oneshot

We are Hiring!


SmartHR